

Reporte: Actividad 3

“Realizar una aplicación con elementos de Jetpack compose”

Alumnos:

- Juan Carlos Malanco Rodriguez
- Isabel Kyra Romero Borunda
- Jocelyn Flores Gutiérrez

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de Aplicaciones Móviles

Fecha: 06/09/2026

● Índice

● Índice.....	2
● Diseño.....	3
● Informe de IA.....	6

Desarrollar una aplicación móvil con Jetpack Compose que integre componentes interactivos, animaciones fluidas y validaciones robustas en formularios, aplicando principios de diseño centrado en el usuario y manejo eficiente de estados.

Repositorio en Github: <https://github.com/JoFloresGutierrez06/Actividad-3---DAM>

Figma:

<https://www.figma.com/design/jGswmEJrjVPI2dljty5AkP/Sin-t%C3%ADtulo?node-id=0-1&t=x4twlOtDg5lNgvjP-1>

● Diseño

Paleta de colores:



Tipografías: tipografía sans-serif similar a Roboto

Títulos de pantalla: tamaño aproximado de 16–18 px, peso Medium.

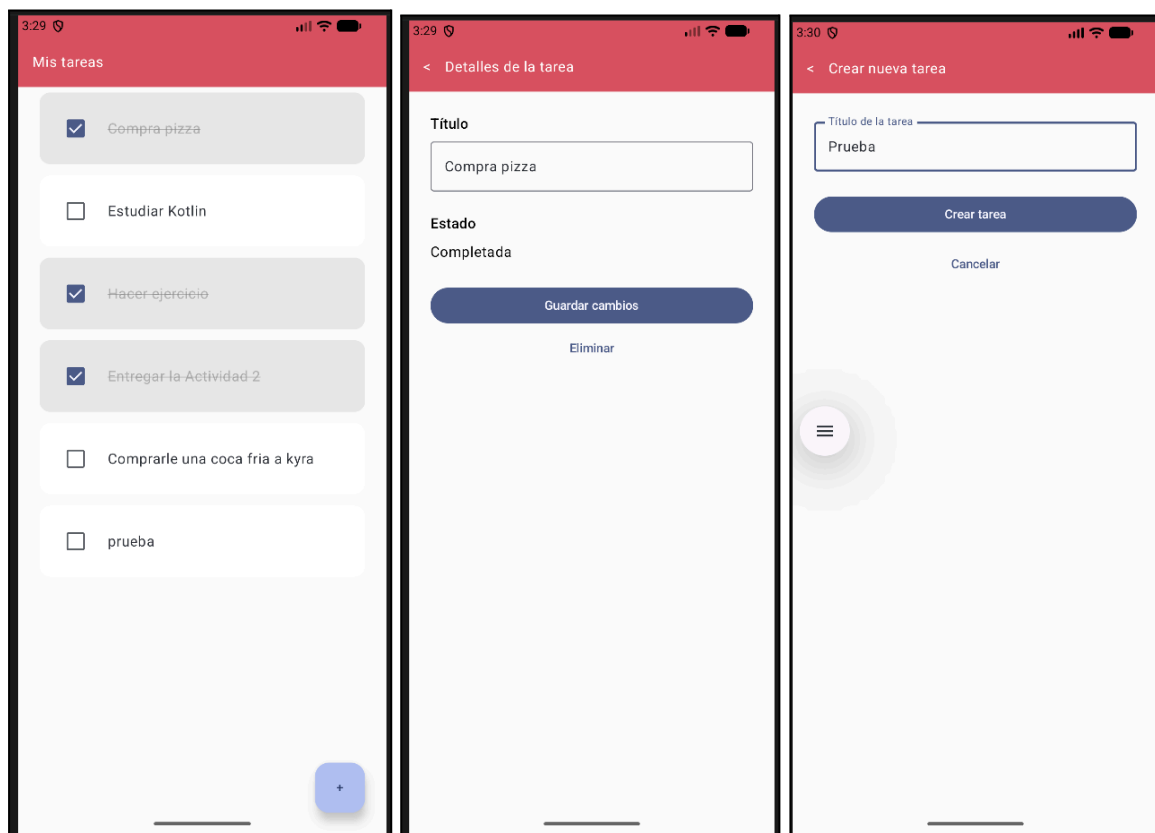
Títulos de tareas: 14–16 px.

Descripción, fecha y textos secundarios: 10–12 px.

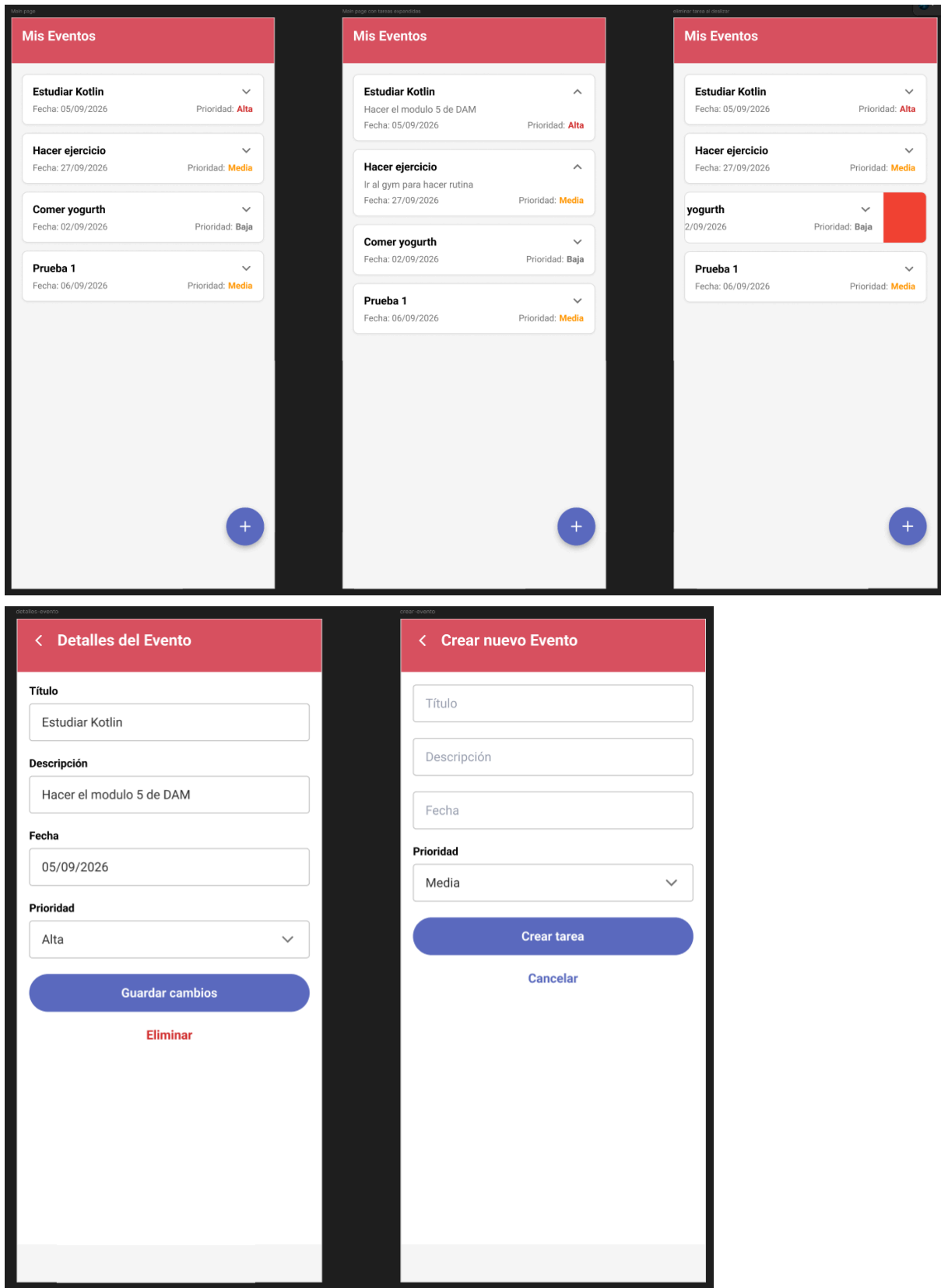
Botones: 12–14 px, peso Medium.

Etiquetas de prioridad: 10–12 px.

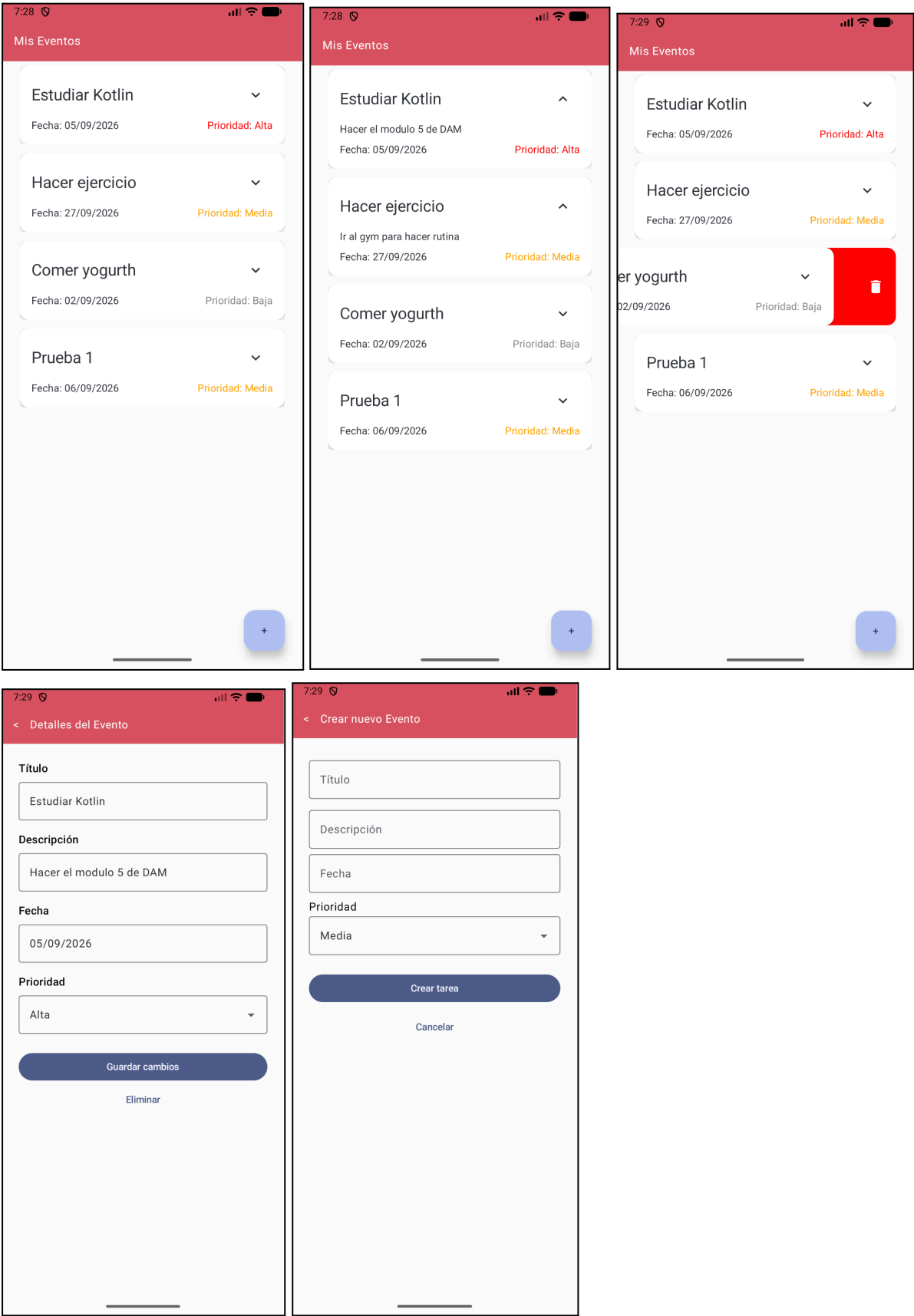
Se trató de mejorar el diseño anterior que se tenía de la aplicación:



Diseño en Figma:



Pantallas finales de la aplicación en Android Studio:



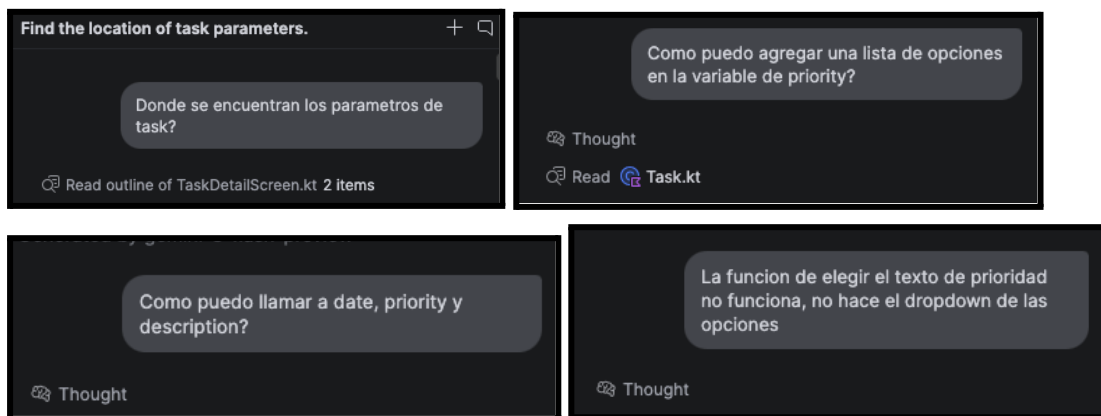
● Informe de IA

● Capturas de prompts:

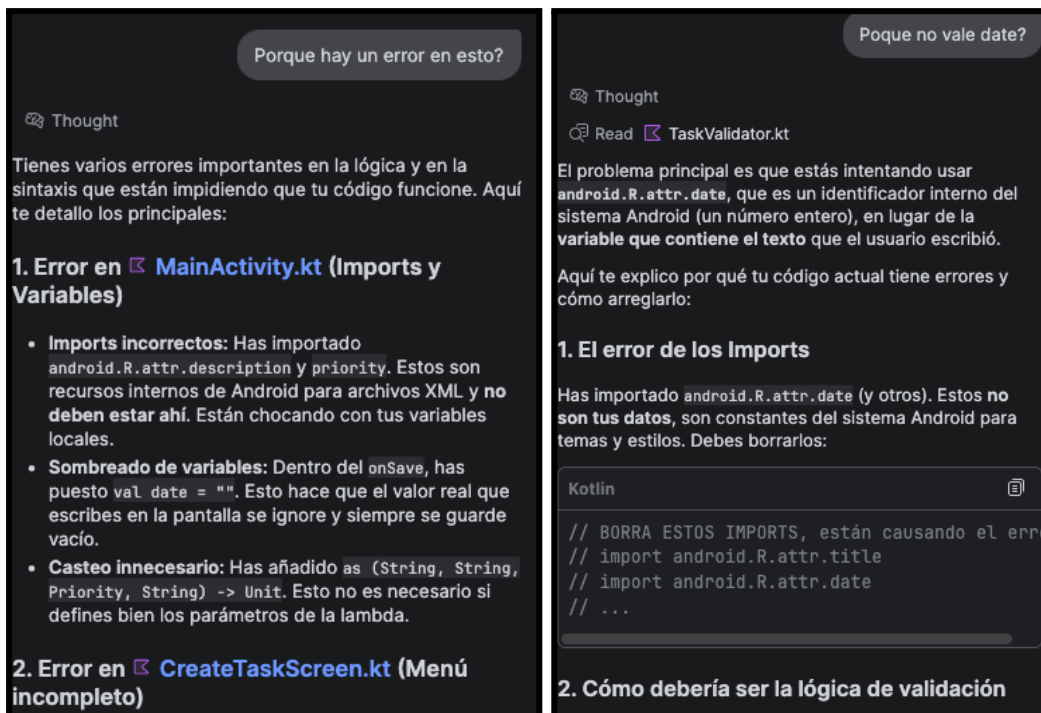
Como se usó la base de la actividad 2 primero se le preguntaron cosas para tener identificado la estructura general del proyecto, así como para entender mejor el código. Luego se le fue preguntando cómo se podían hacer cambios, por ejemplo cómo declarar una variable con opciones, como declarar la variable, que se podía usar en vez de un textfield, etc. Aparte al momento de marcar errores de lo que no se tenía conocimiento se le preguntaba qué es lo que fallaba y nos daba una solución en código. Igual que se le preguntaba cómo se podrían implementar ciertas funciones.

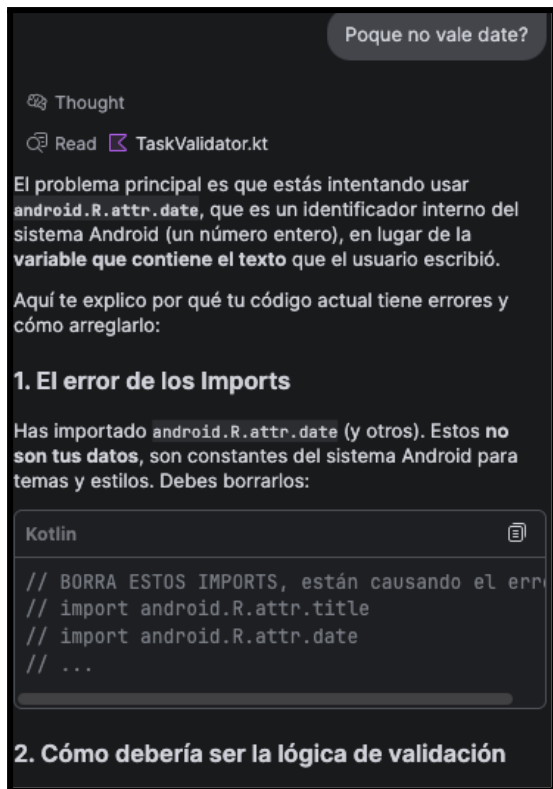
A continuación se van a mostrar algunos de los usos con la ia:

Prompts



Ejemplo de errores que hiban saliendo





- Algunos códigos generados:

Prompt: ¿Cómo puedo agregar una lista de opciones en la variable de priority?
package com.example.actividad1

```
// Definimos las opciones de prioridad
enum class Priority {
    LOW, MEDIUM, HIGH
}
```

```
data class Task(
    val id: Int,
    var title: String,
    var date: String,
    var priority: Priority, // Usamos el enum aquí
    var completed: Boolean // Nota: He mantenido esta variable ya que se usa en otras
    pantallas
)
```

Prompt: ¿Cómo puedo implementar el gesto de swipe to dismiss?

```
// Fragmento de la implementación en TaskListScreen
SwipeToDismissBox(
    state = dismissState,
    backgroundColor = {
```

```

        val color = if (dismissState.dismissDirection == SwipeToDismissBoxValue.EndToStart)
            Color.Red else Color.Transparent
    Box(
        modifier = Modifier.fillMaxSize().background(color, CardDefaults.shape),
        contentAlignment = Alignment.CenterEnd
    ){
        Icon(Icons.Default.Delete, contentDescription = "Eliminar", tint = Color.White)
    }
},
enableDismissFromStartToEnd = false
){
    TaskItem(task = task, ...)
}

```

- **Algunas de las soluciones dadas:**

Solución Completa

Paso 1: Limpiar [MainActivity.kt](#)

Borra los imports de `android.R.attr` y corrige la función crear:

```

Kotlin
// BORRA ESTOS IMPORTS al principio del archivo
// import android.R.attr.description
// import android.R.attr.priority

// ... dentro del NavHost ...
composable("crear") {
    CreateTaskScreen(
        onSave = { title, date, priority, description -> {
            val newId = (tasks.maxOfOrNull { it.id })?.plus(1) ?: 1
            tasks.add(
                Task(
                    id = newId,
                    title = title,
                    date = date, // Usa el parámetro date
                    priority = priority,
                    description = description
                )
            )
            navController.popBackStack()
        }
    },
    onBack = { navController.popBackStack() }
)
}

```

Scroll to bottom

Porque sale error?

Thought

Read [TaskValidator.kt](#)

Thought

Edited [TaskValidator.kt](#) -14 +6

This change introduced 2 warnings, and fixed 1 warning

Tried to edit [CreateTaskScreen.kt](#)

Read [CreateTaskScreen.kt](#)

Edited [CreateTaskScreen.kt](#) -3 +3

This change fixed 1 error, and 1 warning remains from previous

El error ocurre por tres razones técnicas que impiden que tu código compile:

- Parámetros incorrectos:** En [TaskValidator.kt](#) declaraste `isValid` con 4 parámetros, pero en [CreateTaskScreen.kt](#) la estabas llamando con solo 3.
- Conflicto de tipos:** Tu variable `errorMessage` es un `String?` (texto), pero la función `isValid` devolvía un `Boolean` (true/false). No puedes asignar un booleano a una variable de texto.
- Contexto no válido:** Declaraste `isValid` como `@Composable`. Las funciones `Composable` son para dibujar la interfaz, no para realizar validaciones lógicas dentro de un botón (`onClick`). Al intentar dibujar un `Text` dentro de un `onClick`, se falló.

Me puedes explicar como se puede implementar viewModel en este proyecto?

Thought

Para implementar un **ViewModel** en este proyecto, seguiremos el patrón de arquitectura recomendado por Google. El objetivo es sacar toda la "lógica de negocio" (la lista de tareas y las funciones de añadir/editar/borrar) de la **MainActivity** y ponerla en una clase especializada que sobreviva a cambios de configuración (como girar la pantalla).

Aquí tienes los pasos para hacerlo:

- 1. Agregar la dependencia**

Primero, asegúrate de tener la librería de **ViewModel** para Compose en tu archivo `build.gradle.kts` (`Module :app`):

```

Kotlin
dependencies {
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose")
}

```

- 2. Crear el TaskViewModel**

Crea un nuevo archivo llamado `TaskViewModel.kt`. Aquí

Scroll to bottom